Programming Assignment 2

AY 2026 – 2027 / Semester 1

Overview

In this assignment, you will learn to develop your very own data plane programs using the Programming Protocol-Independent Packet Processors (P4) framework in an emulated environment powered by the Mininet network emulator and the BMv2 virtual P4 switch. By working through the tasks, you will learn (1) how to develop, compile and run P4 programs, (2) how to build a P4 controller to control and manage your P4 programs using P4Runtime, and (3) how to leverage P4 to implement new network functionality. **Please read through this document carefully before attempting the tasks.**

Submission Deadline

Nov 6th (Week 12 Friday) 23:59.

Setting Up The Environment

You may either use the provided VM image, or the SoC VM using the installation scripts.

Understanding The Environment

The Mininet repository consists of the following components:

- **common**: Provides the common classes and functions used in the repository.
- **networks**: Defines the Mininet networks and associated configuration files for the tasks.
- **tasks**: Contains the P4 program and controller templates for the tasks.
- **Makefile**: Contains the commands required to manage Mininet, BMv2, and the P4 controller.
- **requirements.txt**: Defines the Python dependencies required by the programs in the repository.

The tasks folder consists of 2 files for each task, namely main.p4 and controller.py. main.p4 contains the template code for the V1Model architecture used by BMv2. The architecture defines the structure of the processing pipelines and affects how the developer programs the switch. On the other hand, controller.py contains the template code that interacts with the P4 program at runtime. The code uses the Finsy framework which provides convenient interfaces to control and manage the switch, eliminating the need to implement gRPC communications and asynchronous functions by ourselves.

For this assignment, **you will only need to modify the main.p4 and controller.py files in the tasks folder** since all other components have been preconfigured for you. However, do consider spending some time to examine the codebase as it will be beneficial for you to understand how it works under the hood.

Working With The Environment

When the Mininet network is executed, it generates a folder named temp. This folder contains two subfolders, p4 and controller.

The p4 subfolder stores the temporary files generated by BMv2. It also contains several log files and packet captures of the ports on the switch. The packet captures are especially useful for debugging issues with your P4 programs. You may view the packet captures using a tool such as Wireshark or TCPDUMP.

The controller subfolder stores the temporary files generated by the controller. It also contains the log file that captures the output from the controller. You may use Finsy's LoggerAdapter class to log additional information to the log file for troubleshooting purposes.

Submission Instructions

You will submit a ZIP file on Canvas under "Programming Assignment 2". To generate the ZIP file for submission, execute the command "**make generate-submission**" and enter your student ID when prompted. The program will produce a ZIP file with your student ID (e.g. A0123456X.zip) in the present working directory. Do check the ZIP file to ensure that the tasks folder containing your code was zipped successfully.

Useful References

The following list contains some references that may be useful to you as you work on this assignment:

- **P4 Language Tutorial**: https://bit.ly/p4d2-2018-spring
- **P4 V1Model**: https://raw.githubusercontent.com/p4lang/p4c/refs/heads/main/p4include/v1model.p4
- **Finsy P4Runtime Controller Library**: https://byllyfish.github.io/finsy/finsy.html

Common Issues and Resolution Steps

Controller throws finsy.p4client.P4ClientError with message "Not primary"

When starting the P4Runtime controllers and the Mininet network, the controllers may raise the following error that can be seen in the log file:

```
CRITICAL finsy [] Switch task 'fy:s1|controller_ready_handler failed
finsy.p4client.P4ClientError: operation=WriteRequest code=GRPCStatusCode.PERMISSION_DENIED
message='Not primary'
```

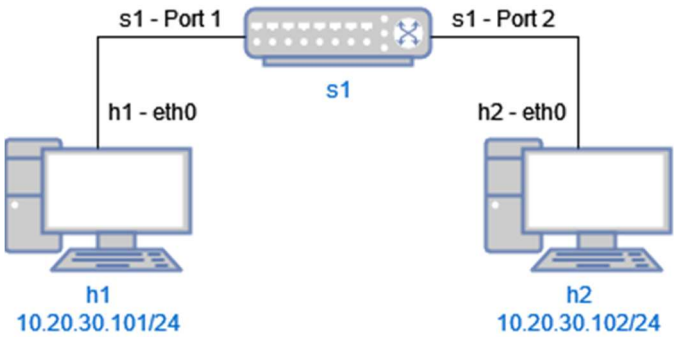
This error indicates that there may be a different instance of the controller running in the background that is conflicting with the current instance. To resolve this error:

1. List the instances of the controllers using “**sudo ps aux | grep -i controller.py**”.
2. Kill any running instances of the controllers using “**sudo kill -9 PID**”, where PID is the process ID of the controller.
3. Clean up the environment using “**make clean**”.
4. Run the Mininet network again using “**make run-task-X**”, where X is the task number.

Task 1: Creating A Transparent Bridge (5%)

In this task, you will create a transparent bridge to allow two hosts connected to the switch to communicate. The objectives of this task are to (1) get started with creating a data plane program using the V1Model architecture and (2) understand the intrinsic metadata provided by the architecture. Please add the code for your P4 program to **tasks/1/p4/main.p4**.

The network topology for this task consists of a single P4 switch connected to two hosts.



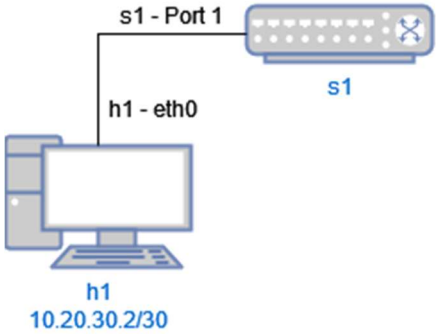
Your task is to create a P4 program for s1 to bridge the connection between h1 and h2. In other words, when s1 receives a frame from h1, s1 will forward the frame to h2 without alterations and vice versa.

You can test your P4 program manually by using the commands "**make run-task-1**" to run the Mininet network and "**h1 ping -c 4 h2**" to verify the connectivity between h1 and h2.

Task 2: Creating An ICMP Responder (20%)

In this task, you will create an ICMP Responder that will respond to ICMP echo requests sent by a host. The objectives of this task are to (1) understand how to define and parse packet headers, (2) explore how to manipulate packets, and (3) understand how to compute and update the checksums for the packets. Please add the code for your P4 program to **tasks/2/p4/main.p4**.

The network topology for this task consists of a single P4 switch connected to a single host.



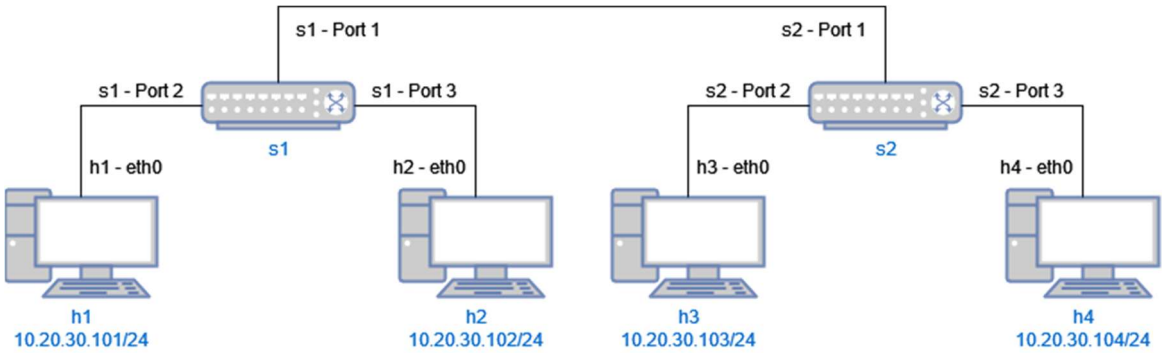
Your task is to create a P4 program for s1 that responds to incoming ICMP echo requests. In other words, when s1 receives an ICMP echo request from h1, s1 will generate the corresponding ICMP echo reply and send it back to h1. For this task, s1 should respond to ICMP echo requests regardless of the destination IP address stated in the request. As long as s1 receives an ICMP echo request on a port, it will generate the corresponding ICMP echo reply and send it back via the same port.

You can test your P4 program manually by using the commands "**make run-task-2**" to run the Mininet network and "**h1 ping -c 4 1.1.1.1**" to verify that the ICMP echo replies are generated correctly. You should be able to ping any unicast address outside the subnet 10.20.30.0/30 and receive a response.

Task 3: Implementing A Layer 2 Hub (15%)

In this task, you will implement a basic Layer 2 Hub to allow hosts on the same subnet to communicate. A network hub is a simple device that connects multiple hosts in a network by broadcasting data to all connected devices, regardless of the destination. The objectives of this task are to (1) understand the Packet Replication Engine (PRE) and (2) get started with creating a P4Runtime controller. Please add the code for your P4 program and controller to **tasks/3/p4/main.p4** and **tasks/3/controller/controller.py** respectively. Note that the same P4 program will be loaded onto all switches when the Mininet network is started.

The network topology for this task consists of two P4 switches connected to four hosts.



Your task is to create a P4 program and controller to implement a network hub and allow all four hosts to communicate with each other. In other words, when the hub receives a frame from one of its ports, it will simply broadcast the frame to all other ports. For this task, you should ensure that your implementation does not inadvertently cause a broadcast storm.

You can test your P4 program manually by using the commands "**make run-task-3**" to run the Mininet network and "**pingall**" to verify the connectivity between all hosts.

Task 4: Implementing A Layer 2 Switch (30%)

This task is a continuation of the previous task. You should copy the codes you have written in task 3 and paste them into task 4 to continue working on them. In this task, you will implement a basic Layer 2 Switch to allow hosts on the same subnet to communicate. A network switch is a device that connects multiple hosts in a network by using a frame's MAC addresses to forward data only to the specific destination device. The objectives of this task are to (1) explore how to use Match-Action Tables and (2) combine everything we've learnt so far to develop a functional network switch. Please add the code for your P4 program and controller to **tasks/4/p4/main.p4** and **tasks/4/controller/controller.py** respectively. Note that the same P4 program will be loaded onto all switches when the Mininet network is started.

The network topology for this task is the same as the topology in the previous task. Your task is to create a P4 program and controller to implement a network switch with MAC address learning capabilities and allow all four hosts to communicate with each other. When a switch first starts, its MAC address tables are empty and it doesn't know which devices are connected to what ports. As the switch begins to receive frames, it records the source MAC address and the ingress port in its MAC address tables. If the tables contain an entry that matches the destination MAC address of a frame, the switch sends the frame to the corresponding port only. However, if there is no matching entry, the switch will broadcast the frame to all ports, ensuring it reaches the intended host. For this task, you should ensure that your implementation does not inadvertently cause a broadcast storm.

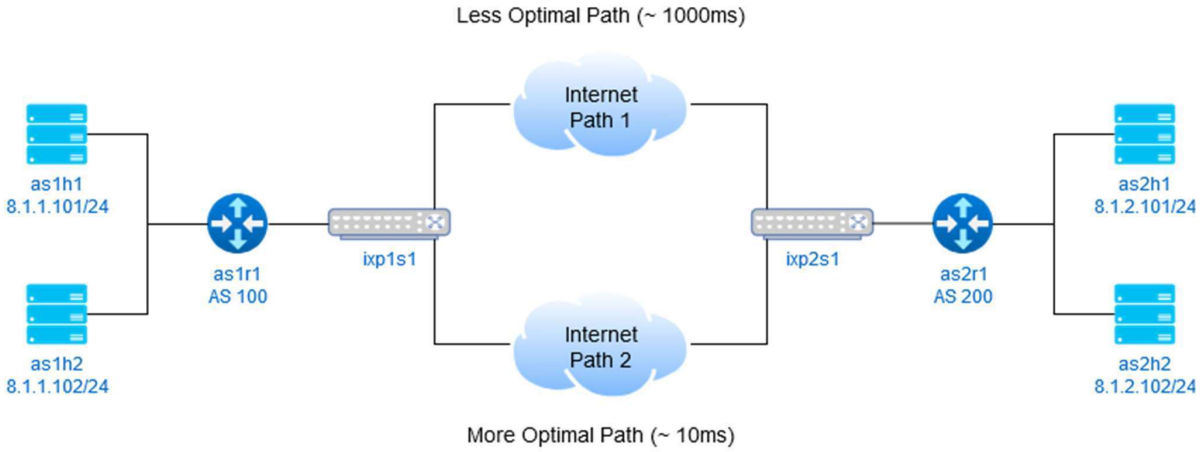
You can test your P4 program manually by using the commands "**make run-task-4**" to run the Mininet network and "**pingall**" to verify the connectivity between all hosts.

Task 5: Implementing A Switch With Custom Network Functionalities (30%)

This task is a continuation of Task 4. You should copy the codes you have written in task 4 and paste them into task 5 to continue working on them. In this task, you will implement a switch with your own custom functionalities to alter the routes taken by selected network traffic.

The objective of this task is to leverage everything we've learnt so far to implement new custom functionalities in the data plane. Please add the code for your P4 program and controller to **tasks/5/p4/main.p4** and **tasks/5/controller/controller.py** respectively. You may create additional files in the **tasks/5/** folder but do not remove or rename the provided template files. Note that the same P4 program will be loaded onto all switches when the Mininet network is started.

The network topology for this task consists of several pre-configured Autonomous Systems (ASes) running the Border Gateway Protocol (BGP).



The routers as1r1 and as2r1 are connected to the switches ixp1s1 and ixp2s1 respectively. Both routers have been pre-configured to peer with the other routers connected to their respective switches using BGP to form Internet Path 1 and Internet Path 2, allowing servers in both AS 100 and AS 200 to communicate with each other. Although the traffic sent by one AS can use either path to reach the other AS, Internet Path 1 is a less optimal path as compared to Internet Path 2. A round-trip from one AS to the other using Internet Path 1 will take around 1000ms whereas a round-trip from one AS to the other using Internet Path 2 will only take around 10ms. By default, the BGP configuration of the routers will cause traffic between AS 100 and AS 200 to take the less optimal Internet Path 1, resulting in a higher round-trip time as seen in the output of the ping command below:

```
mininet> as1h1 ping -c 4 as2h1
PING 8.1.2.101 (8.1.2.101) 56(84) bytes of data.
64 bytes from 8.1.2.101: icmp_seq=1 ttl=61 time=1015 ms
64 bytes from 8.1.2.101: icmp_seq=2 ttl=61 time=1007 ms
64 bytes from 8.1.2.101: icmp_seq=3 ttl=61 time=1009 ms
64 bytes from 8.1.2.101: icmp_seq=4 ttl=61 time=1009 ms

--- 8.1.2.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3011ms
rtt min/avg/max/mdev = 1006.802/1009.858/1014.573/2.870 ms, pipe 2
mininet>
```

Since the route taken by a packet boils down to how BGP is configured and how peering agreements are established, it can result in less-than-ideal routes for latency-sensitive applications as demonstrated above. The goal of this task is to design and implement a mechanism in the switches to override the routing decisions made by BGP and alter the routes taken by certain latency-sensitive network traffic traversing through them without modifying the BGP protocol itself. In the **networks/5/ixp_switch/** folder, you will see 2 JSON configuration files **ixp1s1-route-alterations.json** and **ixp2s1-route-alterations.json** for the switches ixp1s1 and ixp2s1 respectively. These files contain the following information on the network traffic

whose paths we would like to alter:

Key	Value	Description
src_addr	String	Source IP address of the packet
dst_addr	String	Destination IP address of the packet
protocol	Integer	IP Protocol number of the packet
src_port	Integer	Source port of the packet (TCP and UDP only)
dst_port	Integer	Destination port of the packet (TCP and UDP only)
egress_mac	String	MAC address of the router that should receive the packet
egress_port	Integer	Port ID of the switch in which the router that should receive the packet is connected

The src_port and dst_port keys are only present in entries for TCP (protocol 6) and UDP (protocol 17) traffic. Entries for other protocols, such as ICMP (protocol 1), only specify the source address, destination address and protocol of the packet.

For example, the following configuration entry

```
{
    "src_addr": "8.1.1.101",
    "dst_addr": "8.1.2.101",
    "protocol": 6,
    "src_port": 1337,
    "dst_port": 2674,
    "egress_mac": "f0:00:0d:01:04:00",
    "egress_port": 3
}
```

means that if the switch sees a TCP packet originating from host 8.1.1.101 on port 1337 and destined for host 8.1.2.101 on port 2674, it should forward the packet to the router f0:00:0d:01:04:00 connected to port 3 of the switch.

Your task is to build upon your P4 implementation of a Layer 2 Switch developed in Task 4 by adding the route alteration mechanism described above. The new and enhanced switch shall be implemented according to the following requirements:

1. The switch shall parse the configuration files in the **networks/5/ixp_switch/** folder and configure the switch accordingly during initialisation (i.e., the configuration values shall not be hardcoded into the switch).
2. If the switch detects a packet that matches the criteria defined in the configuration file, it should invoke the route alteration mechanism. However, if a packet does not match the criteria, it should be processed normally.
3. The switch shall maintain compliance with the requirements of Task 4.

You can test your P4 program manually by using the commands "**make run-task-5**" to run the Mininet network and "**as1h1 ping -c 4 as2h1**" to check the round-trip time. Note that the routers may take up to a minute after the network starts to establish their BGP sessions, so the first few pings may fail. If the route alteration mechanism is working as intended, you should see that the round-trip time from as1h1 to as2h1 has reduced from about 1000ms to about 10ms. If you would like to perform additional tests to validate your data and control plane programs, you can generate custom packets using the pre-installed Scapy framework.